

1. Write a Python script using sqlite3 to connect to todo_list.db, create the tasks table with the required fields, commit the changes, and close the connection.

```
import sqlite3

conn = sqlite3.connect("todo_list.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    description TEXT NOT NULL,
    due_date TEXT,
    is_completed INTEGER NOT NULL DEFAULT 0
)
""")

conn.commit()

print("Tasks table created successfully.")

conn.close()
```

Output:

```
In [10]: runfile('D:/TyBsc39/TY/Assign5 Q1.py', wdir='D:/TyBsc39/TY')
Tasks table created successfully.
```

2. Write Python functions using sqlite3 to add, view, complete, and delete tasks in the tasks table with error handling and proper connection management.

```
import sqlite3

def add_task(description, due_date):
    try:
        conn = sqlite3.connect("todo_list.db")
        cursor = conn.cursor()

        cursor.execute("""
            INSERT INTO tasks (description, due_date, is_completed)
            VALUES (?, ?, ?)
            """, (description, due_date, 0))

        conn.commit()

        task_id = cursor.lastrowid

        print(f"Task added successfully. Task ID: {task_id}")

        conn.close()

    except sqlite3.Error as e:
        print("Database error:", e)

def view_tasks(show_completed=False):
    try:
        conn = sqlite3.connect("todo_list.db")
        cursor = conn.cursor()

        if show_completed:
            cursor.execute("""
                SELECT id, description, due_date, is_completed
                FROM tasks
                ORDER BY id ASC
                """)
        else:
            cursor.execute("""
                SELECT id, description, due_date, is_completed
                FROM tasks
                WHERE is_completed = 0
                ORDER BY id ASC
                """)

        tasks = cursor.fetchall()

        if not tasks:
            print("No tasks found.")
        else:
            print("\n----- Tasks -----")
```

```
for task in tasks:
    task_id = task[0]
    description = task[1]
    due_date = task[2]
    is_completed = task[3]

    if is_completed == 1:
        status = "Completed"
    else:
        status = "Pending"

    print(
        f"ID: {task_id}, "
        f"Description: {description}, "
        f"Due Date: {due_date}, "
        f"Status: {status}"
    )

conn.close()

except sqlite3.Error as e:
    print("Database error:", e)

def mark_task_completed(task_id):
    try:
        conn = sqlite3.connect("todo_list.db")
        cursor = conn.cursor()

        cursor.execute("""
            UPDATE tasks
            SET is_completed = 1
            WHERE id = ?
            """, (task_id,))

        if cursor.rowcount > 0:
            print("Task marked as completed.")
        else:
            print("Task not found.")

        conn.commit()
        conn.close()

    except sqlite3.Error as e:
        print("Database error:", e)

def delete_task(task_id):
    try:
        conn = sqlite3.connect("todo_list.db")
        cursor = conn.cursor()

        cursor.execute("""
            DELETE FROM tasks
```

```
        WHERE id = ?
        """, (task_id,))

    if cursor.rowcount > 0:
        print("Task deleted successfully.")
    else:
        print("Task not found.")

    conn.commit()
    conn.close()

except sqlite3.Error as e:
    print("Database error:", e)
```

3. Create a Python main program loop using a menu-driven interface to add, view, complete, and delete tasks from a To-Do List Manager.

```
from Assign5_Q2 import add_task, view_tasks, mark_task_completed, delete_task
while True:
    print("\n=====")
    print("  To-Do List Manager")
    print("=====")
    print("1. Add New Task")
    print("2. View Pending Tasks")
    print("3. View All Tasks (including completed)")
    print("4. Mark Task as Completed")
    print("5. Delete Task")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        description = input("Enter task description: ")
        due_date = input("Enter due date (optional): ")

        add_task(description, due_date)

    elif choice == "2":
        view_tasks(False)

    elif choice == "3":
        view_tasks(True)

    elif choice == "4":
        try:
            task_id = int(input("Enter task ID: "))
            mark_task_completed(task_id)

        except ValueError:
            print("Please enter a valid task ID.")

    elif choice == "5":
        try:
            task_id = int(input("Enter task ID: "))
            delete_task(task_id)

        except ValueError:
            print("Please enter a valid task ID.")

    elif choice == "6":
        print("Thank you for using To-Do List Manager.")
        print("Goodbye!")
        break

    else:
        print("Invalid choice. Please try again.")
```

Output:

```

In [13]: runfile('D:/TyBsc39/TY/Assign5 Q3.py', wdir='D:/TyBsc39/TY')

=====
      To-Do List Manager
=====
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit

Enter your choice: 1

Enter task description: Work

Enter due date (optional): 0
Task added successfully. Task ID: 3

=====
      To-Do List Manager
=====
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit

Enter your choice: 2

----- Tasks -----
ID: 2, Description: 2, Due Date: 3, Status: Pending
ID: 3, Description: Work, Due Date: 0, Status: Pending

=====
      To-Do List Manager
=====
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit

Enter your choice: 4

Enter task ID: 3
Task marked as completed.

=====
      To-Do List Manager
=====
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit

Enter your choice: 3

----- Tasks -----
ID: 2, Description: 2, Due Date: 3, Status: Pending
ID: 3, Description: Work, Due Date: 0, Status: Completed

=====
      To-Do List Manager
=====
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit

Enter your choice: 5

Enter task ID: 3
Task deleted successfully.

=====
      To-Do List Manager
=====

```

```
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
```

Enter your choice: 5

Enter task ID: 2

Task deleted successfully.

=====

To-Do List Manager

=====

```
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
```

Enter your choice: 3

No tasks found.

=====

To-Do List Manager

=====

```
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
```

Enter your choice: 6

Thank you for using To-Do List Manager.
Goodbye!

In [14]: